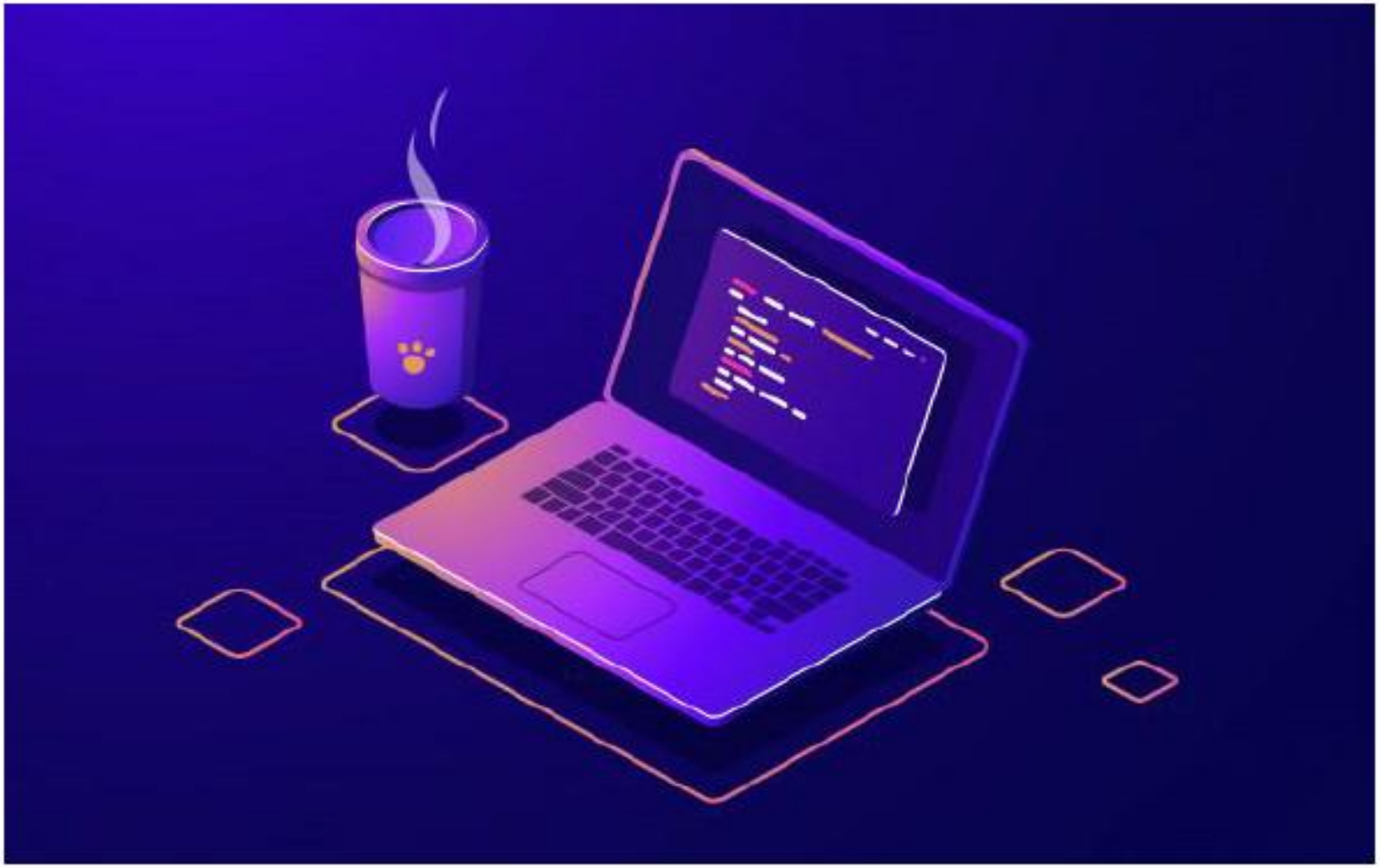


PROGRAMACIÓN

TEMA 4: PROGRAMACIÓN ORIENTADA A OBJETOS. CLASES

POO. CLASES.

iNDICE:

- CREACIÓN DE PAQUETES.
- ESTRUCTURA Y MIEMBROS DE UNA CLASE.
- TRABAJANDO CON MÉTODOS.
- LOS CONSTRUCTORES.
- LOS DESTRUCTORES.
- ENCAPSULACIÓN Y VISIBILIDAD. INTERFACES.
- HERENCIA.

POO. CLASES.

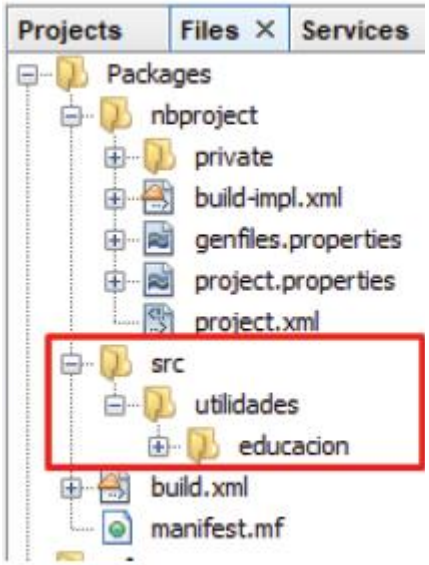
CREACIÓN DE PAQUETES:

Los conceptos *paquete* y *CLASSPATH* los hemos visto brevemente en el tema 2. Sobre los paquetes hay que tener en cuenta lo siguiente:

- Un paquete es un conjunto de clases relacionadas entre sí.
- Un paquete puede contener a su vez subpaquetes.
- Java mantiene su biblioteca de clases en una estructura jerárquica.
- Cuando nos referimos a una clase de un paquete (salvo que se haya importado el paquete) hay que referirse a la misma especificando el paquete (y subpaquete si es necesario) al que pertenece (por ejemplo: java.io.File).
- Los paquetes permiten reducir los conflictos con los nombres puesto que dos clases que se llaman igual, si pertenecen a paquetes distintos, no deberían de dar problemas.
- Los paquetes permiten proteger ciertas clases no públicas al acceso desde fuera del mismo.

POO. CLASES.

CREACIÓN DE PAQUETES:



Para la creación de un paquete muy sencillo vamos a seguir los siguientes pasos:

La idea es crear un paquete con dos clases y llamar a dichas clases desde un programa aparte.

1. Lo primero que hay que hacer es crear en el directorio donde estamos compilando los programas un subdirectorio con nombre, por ejemplo, *utilidades*. En este subdirectorio vamos a tener varios paquetes (serán subpaquetes del paquete utilidades). El primero de ellos se va a llamar *educación* y vamos a tener dentro de él dos clases ya compiladas llamadas saludar y despedirse (saludar.class y despedirse.class).
   Cada subpaquete estará situado en un subdirectorio aparte del subdirectorio utilidades. Como se puede observar en la imagen anterior, se está utilizando el IDE NetBeans. En este directorio (src) se guardan los programas y las clases, y es aquí donde crearemos estos subdirectorios.

POO. CLASES.

CREACIÓN DE PAQUETES:

2. Las clases a crear son las siguientes:

```
package Utilidades.educacion;
import java.io.*;

public class saludar{
    public void saludo(){
            System.out.println("Hola");
    }
}
//** Fin código****

//** Inicio código****
package Utilidades.educacion;
import java.io.*;
public class despedirse{
    public void despedida(){
            System.out.println("Adios");
    }
}
```

package utilidades.educación; Con esta sentencia, ambas clases indican que pertenecen al paquete 'educación'.

POO. CLASES.

CREACIÓN DE PAQUETES:

Una vez que he hecho eso, el siguiente paso será importar el paquete con la sentencia import.

```
import Utilidades.educacion.*;
public class test {
    public static void main(String[] args) {
                        saludar s=new saludar();
                        despedirse d-new despedirse();
                        s.saludo();
                        d.despedida();
    }
}
```

Se crearán dos objetos, uno de cada clase (saludar y despedirse) y se hace una llamada a un método de cada clase para verificar que el paquete funciona correctamente. Si se han realizado estos pasos correctamente la compilación no debería dar ningún error.

POO. CLASES.

CONCEPTO DE CLASE:

En la programación orientada a objetos las clases permiten a los programadores abstraer el problema a resolver ocultando los datos y la manera en la que estos se manejan para llegar a la solución (se oculta la implementación). En un programa orientado a objetos NO se recomienda que desde el mismo programa se acceda directamente a las variables internas de una clase si no es a través de métodos getters y setters (por ejemplo getEdad() o setEdad()), que veremos más adelante.

La abstracción es importante en el análisis y diseño de aplicaciones orientadas a objetos. Su finalidad es crear un conjunto de clases que resuelvan el problema que se está abordando.

POO. CLASES.

CONCEPTO DE CLASE:

Por lo tanto, en la definición de nuestras clases deberemos de cuidar lo siguiente:

- No se deberá tener acceso directo a la estructura interna de las clases. El acceso a los atributos será a través de getters y setters.
- En el supuesto de que haya que modificar el código sin modificar el interfaz con otras clases o programas, esto debería poder hacerse sin tener ninguna repercusión con otras clases o programas. Se busca que las clases tengan un alto grado de cohesión (independencia).

En Java hay varios niveles de acceso a los miembros de una clase:

- ***public*** (acceso publico).
- ***protected*** (acceso protegido).
- ***private*** (acceso privado).
- ***no especificado*** (acceso en su paquete).

POO. CLASES.

CONCEPTO DE CLASE:

Cuando especificamos el nivel de acceso a un atributo o método de una clase, lo que estamos especificando es el nivel de accesibilidad que va a tener ese atributo o método que puede ir desde el acceso más restrictivo (private) al menos restrictivo (public).

Una subclase es una clase que hereda ciertas características de la clase padre y que puede añadir algunas propias.

POO. CLASES.

CONCEPTO DE CLASE:

Dependiendo de la finalidad de la clase, utilizaremos un tipo de acceso u otro.

- **Acceso público (public).** Un miembro público puede ser accedido desde cualquier otra clase o subclase que necesite utilizarlo. Una interfaz de una clase estará compuesta por todos los miembros públicos de la misma.
- **Acceso privado (private).** Un miembro privado puede ser accedido solamente desde los métodos internos de su propia clase. Otro acceso será denegado.
- **Acceso protegido (protected).** El acceso a estos miembros es igual que el acceso privado. No obstante, para las subclases o clases del mismo paquete (package) a la que pertenece la clase, se considerarán estos miembros como públicos.
- **Acceso no especificado (paquete)**. Los miembros no etiquetados podrán ser accedidos por cualquier clase perteneciente al mismo paquete.

POO. CLASES.

CONCEPTO DE CLASE:

Modificador de acceso	public	protected	private	Sin especificar (acceso paquete)
¿El método o atributo es accesible desde la propia clase?	sí	sí	sí	sí
¿El método o atributo es accesible desde Otras clases en el mismo paquete?	sí	sí	NO	sí
¿El método o atributo es accesible desde una subclase en el mismo paquete?	sí	sí	NO	sí
¿El método o atributo es accesible desde subclases en otros paquetes?	sí	(*)	NO	NO
¿El método o atributo es accesible desde otras clases en otros paquetes?	sí	NO	NO	NO

POO. CLASES.

CONCEPTO DE CLASE:

2.1 CONTROL DE ACCESO A UNA CLASE:

Cuando creamos una clase en Java es posible definir la relación que esa clase tiene con otras clases o la relación que tendrá esa clase con las clases de su mismo paquete.

Clase pública	Clase NO definida como pública

`public class miClase {`
`.....`
`}`
	`class miClase {`
	`.....`
	`}`

Puede ser utilizada por cualquier clase.	Puede ser utilizada SOLO por clases de su propio paquete.

POO. CLASES.

CONCEPTO DE CLASE:

2.2 REFERENCIA A OBJETO THIS:

Java, al igual que C++, proporciona una referencia al objeto con el que se está trabajando. Esta referencia se denomina this, que no es ni más ni menos que el objeto que está ejecutando el método. En los ejemplos que hemos estado utilizando en muchas ocasiones se obviaba esta referencia puesto que se sobreentiende que el objeto está invocando al método. En algunas ocasiones nos va a servir para resolver ambigüedades o para devolver referencias al propio objeto. En el siguiente ejemplo se ve claramente el uso del this.

En el siguiente código vas a poder apreciar que la referencia *this* en ocasiones se puede omitir. Observa también cómo se devuelve una referencia al propio objeto en los métodos incrementarAncho() e incrementarAlto().

POO. CLASES.

CONCEPTO DE CLASE:

2.2 REFERENCIA A OBJETO THIS:

```
public class rectangulo implements Cloneable
{
    private int ancho = 0;
    private int alto = 0;
    private String nombre;

    rectangulo(int anc, int alt){
        ancho = anc;
        this.alto = alt;
    }
    public int getAncho(){return this.ancho;}
    public int getAlto(){return alto;}


    public int incrementarAncho(){
        ancho++;
        return this.ancho;
    }
    public rectangulo incrementarAlto(){
        alto++;
        return this;
    }
```

POO. CLASES.

CONCEPTO DE CLASE:

2.2 CLASE *OBJECT*:

La clase Object es la raíz jerárquica de Java. Cualquier clase implementada en Java siempre va a ser una subclase de la clase Object. Eso quiere decir que va a heredar todos los métodos de Object. De todos los métodos de la clase Object vamos a ver con más profundidad los siguientes:

Tabla 4.2. Métodos de la clase Object

Método	Descripción
clone()	Permite "clonar" un objeto.
equals()	Permite comparar un objeto con otro.
toString()	Devuelve el nombre de la clase.
finalize()	Método invocado por el recolector de basura (garbage collector) para borrar definitivamente el objeto.

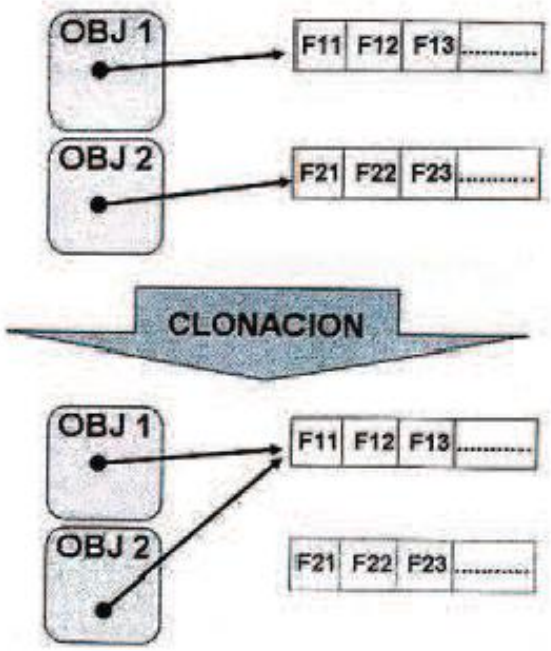
POO. CLASES.

CONCEPTO DE CLASE:

2.2 CLASE *OBJECT*:

Método clone()

El método clone nos permite copiar un objeto en otro. Utilizar este método equivaldría a utilizar un constructor de copia. La clase base Object tiene el método clone() que es el mecanismo que utiliza Java para clonar objetos. Es posible y en muchos casos necesario implementar un método clone, el cual sobrescribirá al método clone de su superclase y podrá actuar de una forma más específica que el método genérico clone().



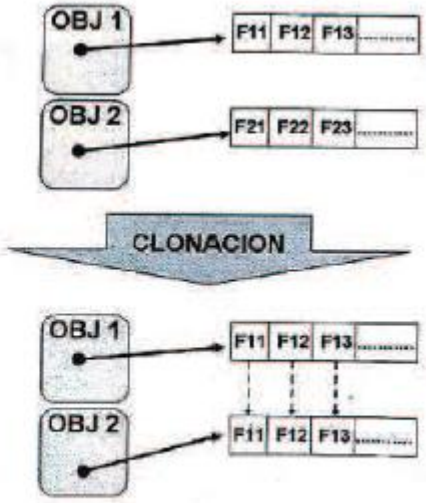
POO. CLASES.

CONCEPTO DE CLASE:

2.2 CLASE *OBJECT*:

El método genérico clone() hace una copia superficial del objeto.

Como se puede ver en la figura anterior, la copia superficial únicamente hace una copia del contenido de un objeto en otro, lo que en algunas ocasiones provoca que la modificación del contenido de un objeto implique el cambio en el clonado y viceversa.



Por el contrario, las copias en profundidad pueden hacer una copia selectiva del contenido de un objeto en otro. En este caso ambos objetos vivirán "vidas independientes".

Un ejemplo de realizar una clonación de un objeto en Java sería el siguiente:

POO. CLASES. CONCEPTO DE CLASE:

2.2 CLASE *OBJECT -> clone()*:

1

```java
// Ejemplo: uso de clone() en Java
class Direccion {
    String ciudad;

    public Direccion(String ciudad) {
        this.ciudad = ciudad;
    }

    @Override
    public String toString() {
        return ciudad;
    }
}

class Persona implements Cloneable {
    String nombre;
    int edad;
    Direccion direccion;

    public Persona(String nombre, int edad,
Direccion direccion) {
        this.nombre = nombre;
        this.edad = edad;
        this.direccion = direccion;
    }
```

2

```java
    // Copia superficial (shallow copy)
    @Override
    public Persona clone() {
        try {
            return (Persona) super.clone();
        } catch (CloneNotSupportedException e) {
            throw new AssertionError();
        }
    }

    // Copia profunda (deep copy)
    public Persona cloneProfundo() {
        try {
            Persona copia = (Persona) super.clone();
            copia.direccion = new Direccion(this.direccion.ciudad);
            return copia;
        } catch (CloneNotSupportedException e) {
            throw new AssertionError();
        }
    }

    @Override
    public String toString() {
        return nombre + " (" + edad + " años, " + direccion + ")";
    }
}
```

3

```java
public class EjemploClone {
    public static void main(String[] args) {
        Direccion dir = new Direccion("Madrid");
        Persona p1 = new Persona("Ana", 30, dir);

        // Copia superficial
        Persona p2 = p1.clone();

        // Copia profunda
        Persona p3 = p1.cloneProfundo();

        // Modificamos la dirección del clon superficial
        p2.direccion.ciudad = "Barcelona";

        // Mostramos los resultados
        System.out.println("Original: " + p1);
        System.out.println("Clon superficial: " + p2);
        System.out.println("Clon profundo: " + p3);
    }
```

Original: Ana (30 años, Barcelona)
Clon superficial: Ana (30 años, Barcelona)
Clon profundo: Ana (30 años, Madrid)

POO. CLASES.

CONCEPTO DE CLASE:

2.2 CLASE *OBJECT*:

Método equals()

El método **equals** permite realizar una comparación entre un objeto y otro. Lo que hace es comprobar que ambas referencias sean iguales, con lo cual no obtenemos más ventaja que con el operador ==. No hace una comparación en profundidad sino que se limita a comprobar las referencias de los objetos. Si se quiere realizar una comprobación en profundidad habrá que reescribir este método.

Un ejemplo de utilización de este método es el siguiente:

```
rectangulo r1 = new rectangulo(5,7);
rectangulo r2 = new rectangulo(5,7);
rectangulo r3 = r1;
if (r1.equals(r2)){
        System.out.println(«Iguales r1 y r2(equals)»);
}
if (r1.equals(r3)){
        System.out.println(«Iguales r1 y r3(equals)»);
}
```

POO. CLASES.

CONCEPTO DE CLASE:

2.2 CLASE *OBJECT*:

Método toString()

El método **toString()** permite obtener el nombre de la clase desde el cual fue invocado. Además del nombre de la clase, devuelve el carácter '@' y la representación hexadecimal del código **hash** del objeto. Un ejemplo de la llamada a este método es el siguiente:

```
rectangulo r1 = new rectangulo(5,7);
rectangulo r2 = new rectangulo(5,7);
rectangulo r3 = r1;
System.out.println(r1.toString());
System.out.println(r2.toString());
System.out.println(r3.toString());
```

Este código devolverá por pantalla lo siguiente:

```
rectangulo@19821f
rectangulo@addbf1
rectangulo@19821f
```

POO. CLASES.

CONCEPTO DE CLASE:

2.2 CLASE *OBJECT*:

Como podemos observar en el código, al hacer r3 = r1 lo que hacemos es que ambas referencias apunten al mismo objeto con lo cual al invocar al método toString() el resultado será el mismo.

Método finalize()

Cuando el recolector de basura de Java (garbage collector) tiene constancia de que no existen más referencias a un objeto concreto, invoca a este método y se encarga de liberar su memoria ocupada. Si el programador necesita realizar una acción una vez destruido un objeto deberá reescribir este método.

POO. CLASES.

ESTRUCTURAS Y MIEMBROS DE UNA CLASE:

3.1 MIEMBROS ESTÁTICOS DE UNA CLASE: *STATIC.*

En Java no existen variables globales, por lo tanto, si queremos utilizar una variable única y que puedan utilizar todos los objetos de una clase deberemos de declararla como estática (static).

A diferencia de los miembros normales o miembros de instancia, los miembros de clase tienen la cláusula static y todos los objetos de la misma clase compartirán dichos miembros.

Veamos como funcionan los atributos estáticos de una clase con el siguiente ejemplo:

```
public class cohete{
    private static int numcohetes=0;
    cohete(){  numcohetes++; }
    public int getcohetes(){ return numcohetes;}
}
```

POO. CLASES.

ESTRUCTURAS Y MIEMBROS DE UNA CLASE:

3.1 MIEMBROS ESTÁTICOS DE UNA CLASE: *STATIC.*

Tenemos una clase la cual tiene un miembro estático. Esta variable numcohetes almacenará el número de objetos cohete que se van creando.

```
public class testestaticos {
    public static void main(String[] args) {
            cohete c1 = new cohete();
            cohete c2 = new cohete();
            cohete c3 = new cohete();
            System.out.println(c1.getcohetes());
            System.out.println(c3.getcohetes());
    }
}
```

Cuando desde otra clase, por ejemplo la anterior, se crean varios objetos de la clase cohete (3 objetos) y se llama al método getcohetes(), ¿qué valores devolverá dicho método?

```
System.out.println(cl.getcohetes());
System.out.println(c3.getcohetes());
```

POO. CLASES.

ESTRUCTURAS Y MIEMBROS DE UNA CLASE:

3.1 MIEMBROS ESTÁTICOS DE UNA CLASE: *STATIC.*

La solución es 3 en ambas llamadas. La variable numcohetes se inicializa a 0 (cero) solo una vez (cuando se crea el objeto c1). Cuando se crean los objetos c2 y c3 no se vuelve a inicializar pues ya existe y es estática, solo se incrementa.

Al haber definido numcohetes como private, no es posible desde nuestra clase testestaticos acceder a c1.numcohetes.

Los miembros o atributos de instancia son aquellos que no son static.

POO. CLASES.

ESTRUCTURAS Y MIEMBROS DE UNA CLASE:

3.2 MÉTODOS DE INSTANCIA Y DE CLASE.

Los métodos de una clase son una abstracción del comportamiento de la misma. Los algoritmos formarán parte de los métodos y contendrán la lógica de la aplicación que queramos desarrollar.

Podemos dividir los métodos en dos bloques:

- Métodos de instancia. Son aquellos utilizados por la instancia.
- Métodos de clase. Son aquellos comunes para una clase. Un método por clase.

Miembros o atributos de instancia y de clase son análogos a métodos de instancia y de clase.

POO. CLASES.

ESTRUCTURAS Y MIEMBROS DE UNA CLASE:

3.3 MÉTODOS DE INSTANCIA.

Los métodos de instancia son, por así decirlo, los llamados métodos "comunes". Cada instancia u objeto tendrá sus propios métodos independientes del mismo método de otro objeto de la misma clase.

```
public class cuadrado{
    private int lado;
    cuadrado(int l){ this.lado = l; }
    public int getArea(){ return lado*lado; }
}
```

En el anterior ejemplo podemos ver un método de instancia.

Los métodos de instancia pueden acceder a los miembros de instancia y también a los miembros de clase.

POO. CLASES.

ESTRUCTURAS Y MIEMBROS DE UNA CLASE:

3.3 MÉTODOS DE INSTANCIA.

La siguiente clase, atendiendo a la regla anterior compilará sin problemas:

```
class test {
    public static int var;
    public int var2;
    public void prueba(){
        var = 3;
        var2 = 5;
    }
}
```

No obstante en vez de utilizar la línea: `var = 3;`

Quizás habría sido más correcto utilizar la siguiente: `test.var = 3;`

La llamada a un método de instancia sería la siguiente:

```
test t = new test();

t.prueba();
```

POO. CLASES.

ESTRUCTURAS Y MIEMBROS DE UNA CLASE:

3.4 MÉTODOS ESTÁTICOS O DE CLASE.

Recuerda las siguientes reglas:

1. Los métodos *static* no tienen referencia this.

2. Un método *static* no puede acceder a miembros que no sean static.

3. Un método *no static* puede acceder a miembros static y no static.

Veamos alguna de estas reglas en un pequeño programa:

```
public class Test {
    public int dato=0;
    public static int datostatico=0;
    public void metodo(){this.datostatico++;}
    public static void metodostatico(){
        this.datostatico++; // Esto da error al compilar
        datostatico++;
    }
    public static void main(String[] args) {
        dato++;  // Esto da error al compilar
        datostatico++;
        metodostatico();
        metodo();  // Esto da error al compilar
    }
}
```

POO. CLASES.

ESTRUCTURAS Y MIEMBROS DE UNA CLASE:

3.4 MÉTODOS ESTÁTICOS O DE CLASE.

Veremos las razones por las cuales las líneas resaltadas en negrita dan error de compilación.

```
this.datostatico++; // Esto da error al compilar
```

La sentencia anterior produce un error debido a la regla 1 (los métodos static no tienen referencia this).

```
dato++; // Esto da error al compilar
```

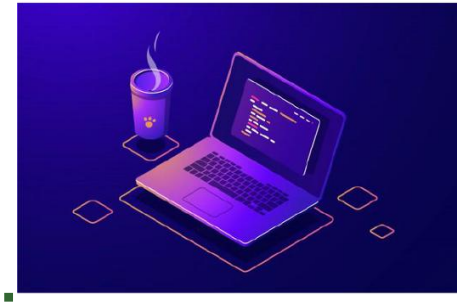
La sentencia anterior produce un error debido a la regla 2 (un método static no puede acceder a miembros que no sean static) dado que dato no es static.

```
metodo(); // Esto da error al compilar
```

La sentencia anterior produce un error debido a la regla 2 (un método static no puede acceder a miembros que no sean static) dado que el método metodo() no es static. Sin embargo según la regla 3, el método metodo() puede acceder al dato *datostatico* dado que éste método no es estático.

Los métodos de clase o static NUNCA pueden acceder a los miembros de instancia.

POO. CLASES.



ESTRUCTURAS Y MIEMBROS DE UNA CLASE:

3.4 MÉTODOS ESTÁTICOS O DE CLASE.

Tabla resumen:

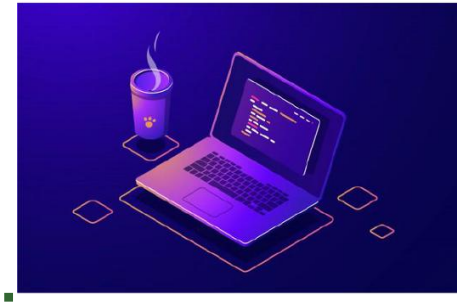
Método	Llamada	Declaración	Acceso
Clase	Clase.metodo(parámetros)	static	Miembros de clase.
Instancia	Instancia.metodo(parámetros)		Miembros de clase y de instancia.

Un ejemplo de los métodos de clase son las funciones de la librería java.lang.Math las cuales pueden ser llamadas anteponiendo el nombre de la clase Math. Un ejemplo de llamada a una de estas funciones es por ejemplo:

```
Math.cos(angulo);
```

Como se puede observar se antepone el nombre de la clase (Math) al del método cos.

Algunos métodos estáticos (los más utilizados) de la clase Math son los siguientes:



POO. CLASES.

ESTRUCTURAS Y MIEMBROS DE UNA CLASE:

3.4 MÉTODOS ESTÁTICOS O DE CLASE.

Método	Descripción
static int abs(int a) static long abs(long a) static double abs(double a) static float abs(float a)	Devuelve el valor absoluto del parámetro pasado.
static int max(int a, int b) static long max(long a, long b) static double max(double a, double b) static float max(float a, float b)	Devuelve el mayor de los valores a ó b.
static int min(int a, int b) static long min(long a, long b) static double min(double a, double b) static float min(float a, float b)	Devuelve el menor de los valores a ó b.
static double pow(double a, double b)	Potencia de un número. Devuelve el valor de a elevado a b.
static double random()	Números aleatorios. Devuelve un número aleatorio de tipo double entre cero y uno (éste último no incluido).
static int round(float a) static long round(double a)	Redondeo. Redondea el parámetro a al valor entero más cercano.

Adicionalmente, la clase Math ofrece un gran número de funciones trigonométricas, además de muchas otras funciones.

POO. CLASES.

TRABAJANDO CON MÉTODOS:

4.1 PASO DE PARÁMETROS POR VALOR Y POR REFERENCIA.

Generalmente es común pasar parámetros a los métodos salvo que sean métodos para inicializar o finalizar el objeto. Existen ocasiones en las que necesitamos que estas variables que pasamos como parámetros cambien su valor una vez ejecutado el método si este las ha modificado. Si es una variable se puede solucionar con la sentencia return, pero imagínate que queremos pasar 5 variables a un método y conservar los valores si éste los modifica. En ese caso con return solamente podríamos obtener una variable modificada. Por lo tanto, deberemos utilizar paso de parámetros por referencia.

- ***Paso de parámetros por valor.*** Los parámetros se copian en las variables del método. Las variables pasadas como parámetro no se modifican.
- ***Paso de parámetros por referencia.*** Las variables pasadas como parámetro se modifican puesto que el método trabaja con las direcciones de memoria de los parámetros.

POO. CLASES.

TRABAJANDO CON MÉTODOS:

4.1 PASO DE PARÁMETROS POR VALOR Y POR REFERENCIA.

```
public class TestParam {
    1 usage
    public static void cambiar (int x){
        x++;
    }
    1 usage
    public static void cambiar1 (int[] x1){
        x1[0]++;
    }
    public static void main (String [] args){
        int x = 3;
        int []x1 = {3};
        cambiar(x);
        System.out.println("El valor de x es: " + x);
        cambiar1(x1);
        System.out.println("El valor de x1[0] es: " + x1[0]);
    }
}
```

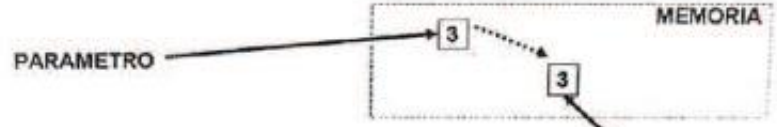
POO. CLASES.

TRABAJANDO CON MÉTODOS:

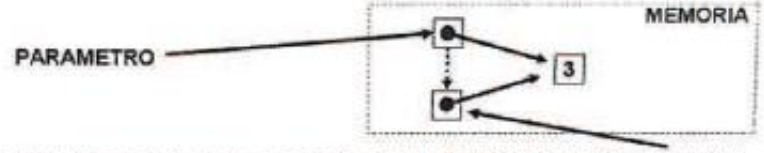
4.1 PASO DE PARÁMETROS POR VALOR Y POR REFERENCIA.

Este programa dará como salida los valores 3 y 4. En la función *cambiar2* se pasa un array en vez de una variable.

La diferencia entre un array de enteros y una variable entera es que un array de enteros es una dirección de memoria donde de manera consecutiva se almacenarán una serie de valores enteros. Los arrays o vectores se estudiarán más adelante en el Tema 8.




```
public static void cambiar(int x){
    x++;
}
```




```
public static void cambiar2(int[] par){
    par[0]++;
}
```

POO. CLASES.

TRABAJANDO CON MÉTODOS:

4.2 MÉTODOS RECURSIVOS.

Un método se llama **recursivo** cuando **se llama a sí mismo**.

¿Cuándo utilizar la recursividad?

- Cuando la resolución de un problema es más sencilla.
- Cuando no es infinita, es decir, hay un caso resoluble más básico o más sencillo.

Generalmente, cuando se va a resolver un problema recursivo vemos que en cada llamada sucesiva al método recursivo nos vamos acercando cada vez más a la solución.

POO. CLASES.

TRABAJANDO CON MÉTODOS:

4.2 MÉTODOS RECURSIVOS. *Ejemplo*

Vamos a ver la recursividad con un ejemplo sencillo. El método que vamos a escoger es la potencia de un número. Nuestro método será el siguiente:

potencia(x, y) --> x^y

Para resolver un caso recursivo generalmente debemos encontrar:

1. Una fórmula o proceso que reduzca la complejidad y nos vaya acercando a la solución.
2. Un caso base que hace que nuestra recursividad no sea infinita.

En el caso de la potencia:

- Sabemos que $x^y = x * x^{y-1}$ (ésta es la fórmula que reduce la complejidad).
- Y que $x^0 = 1$ (caso base).

POO. CLASES.

TRABAJANDO CON MÉTODOS:

4.2 MÉTODOS RECURSIVOS. *Ejemplo*

```
//Método recursivo:
1 usage
public static int potencia(int x, int y){
    if (y==1){
        return x;
    } else{
        return x*potencia(x, y: y-1);
    }
}
//Método no recursivo:
no usages
public static int potencia1(int x, int y){ //x^y -> x=3, y=4 -> 3^4 -> 3*3*3*3
    int m = 1;
    for(int i=1; i<=y; i++){
        m = m*x;
    }
    return m;
}
```

POO. CLASES.

TRABAJANDO CON MÉTODOS:

4.2 MÉTODOS RECURSIVOS. *Ejemplo*

```
public static int potencia(int x, int y){
    if (y == 1){
        return x;
    }else{
        return x * potencia(x,y-1);
    }
}
```

1 X=2 Y=3

=2*4=8

```
public static int potencia(int x, int y){
    if (y == 1){
        return x;
    }else{
        return x * potencia(x,y-1);
    }
}
```

2 X=2 Y=2

=2*2=4

```
public static int potencia(int x, int y){
    if (y == 1){
        return x;
    }else{
        return x * potencia(x,y-1);
    }
}
```

3 X=2 Y=1

POO. CLASES.

LOS CONSTRUCTORES:

Java, al igual que hace con las variables, cuando va a crear un objeto, lo que hace es reservar espacio en memoria para dicho objeto. En esta fase de construcción del objeto, Java crea un constructor público por defecto del objeto. No obstante, si el programador lo considera oportuno se puede crear un constructor diferente que satisfaga las necesidades de la clase.

El constructor se llama de forma automática siempre que se crea un objeto de una clase.

Por lo tanto tenemos dos tipos de constructores:

POO. CLASES.

LOS CONSTRUCTORES:

- **Constructor por defecto.** Cuando no se especifica en el código. Se ejecuta siempre de manera automática e inicializa el objeto con los valores especificados o predeterminados del sistema.
- **Constructor definido.** Puede ser más de uno. Tiene el mismo nombre de la clase. Nunca devuelve un valor y no puede ser declarado como static, final, native, abstract o synchronized. Por regla general se declaran los constructores como públicos (public) para que puedan ser utilizados por cualquier otra clase.

Cuando existe más de un constructor para una clase se dice que este está sobrecargado.

POO. CLASES.

LOS CONSTRUCTORES:

¿Qué hace Java cuando tiene que cargar una clase?

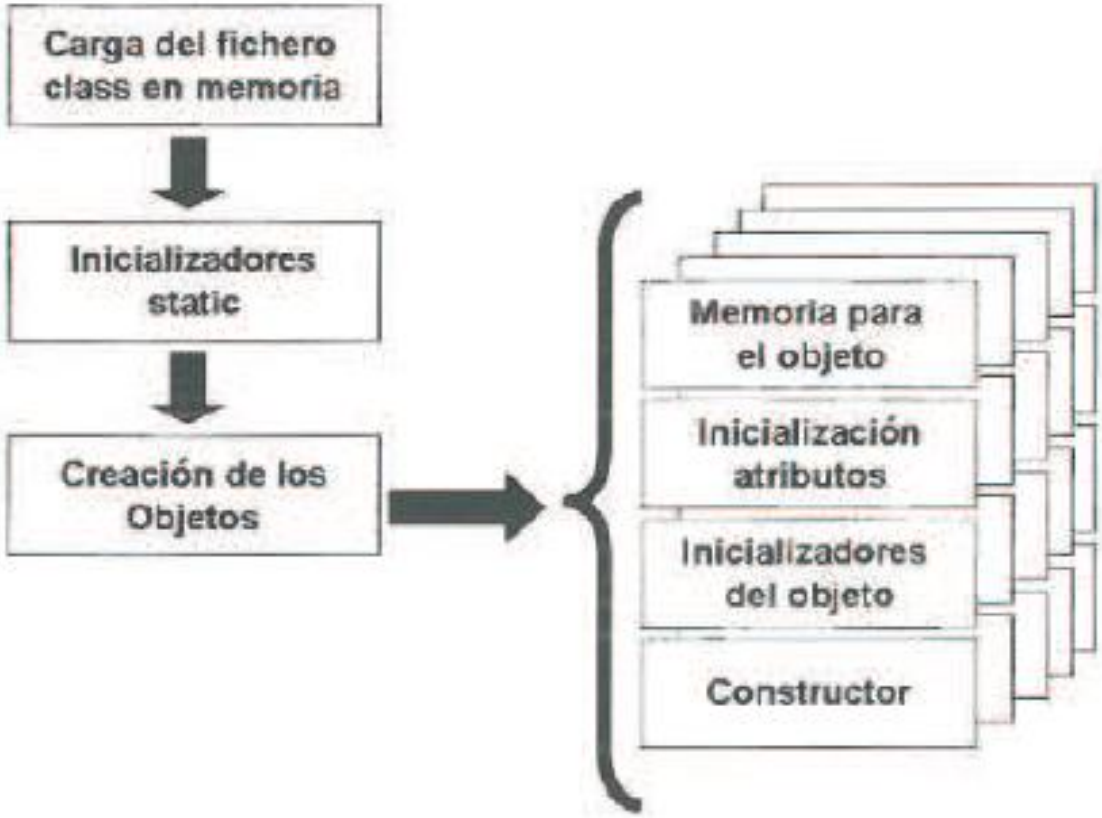
1. Antes de crear el primer objeto, Java localiza el fichero de la clase en disco (recuerda el fichero .class) y lo carga en memoria.
2. Se ejecutarán los inicializadores static de la clase (se explican en la sección a fondo al final de este apartado).
3. Se crea el objeto.

¿Qué hace Java cuando se crea un objeto?

1. Crea memoria para el objeto mediante el operador new.
2. Inicializa los atributos del objeto (solamente los que no fueron inicializados).
3. Se ejecutan los inicializadores de objeto.
4. Llama al constructor adecuado.

POO. CLASES.

LOS CONSTRUCTORES:



POO. CLASES.

LOS CONSTRUCTORES:

5.1 SOBRECARGA DE CONSTRUCTORES.

¿Cuándo necesitamos sobrecargar o definir múltiples constructores para una clase?

Se definen múltiples constructores para una clase cuando el objeto pueda ser inicializado de múltiples formas. Al sobrecargar un constructor variaremos el tipo y número de parámetros que recibe.

```
public class rectangulo {
    private int ancho;
    private int alto;
    rectangulo(int anc, int alt){
        this.ancho = anc;
        this.alto = alt;
    }
    rectangulo(){
        ancho = alto = 0;
    }
    rectangulo(int dato){
        ancho = alto = dato;
    }
//      . . .
```

POO. CLASES.

LOS CONSTRUCTORES

5.1 SOBRECARGA DE CONSTRUCTORES.

Cuando existe más de un constructor para una clase se dice que este está sobrecargado. Cuando creamos un objeto con new, Java elige el constructor más adecuado dependiendo de los parámetros utilizados.

Como según el ejemplo de la clase rectángulo tenemos tres constructores, la creación de cada uno de los objetos siguientes se realizará con un constructor diferente:

```
rectangulo r1 = new rectangulo(5,7);
rectangulo r2 = new rectangulo();
rectangulo r3 = new rectangulo(8);
```

POO. CLASES.

LOS CONSTRUCTORES:

5.2 ASIGNACIÓN DE OBJETOS.

Cuando trabajamos con objetos estamos trabajando con referencias. Una referencia es una localización de la memoria donde se encuentra el objeto.

Como ya se ha dicho, hay que tener en cuenta la utilización de estas referencias cuando se trabaja con objetos. De forma sencilla se va a comprender qué es este enigma de las referencias.

Imaginemos que tenemos la siguiente clase rectángulo:

POO. CLASES.

LOS CONSTRUCTORES:

5.2 ASIGNACIÓN DE OBJETOS.

```
public class rectangulo {
    private int ancho;
    private int alto;
    rectangulo(int anc, int alt){this.ancho = anc; this.alto = alt;}
    rectangulo(){ancho = alto = 0;}
    rectangulo(int dato){ancho = alto = dato;}

    public int getAncho(){return ancho;}
    public int getAlto(){return alto;}
    public int incrementarAncho(){
        ancho++;
        return ancho;
    }
    public rectangulo incrementarAlto(){
        this.alto++;
        return this;
    }
    public static void main(String[] args){
        rectangulo r1 = new rectangulo(5,7);
        rectangulo r2 = new rectangulo();
        r2=r1;
        r2.incrementarAlto();
        r2.incrementarAncho();
        System.out.println("Ancho= "+r1.getAncho());
        System.out.println("Alto= "+r1.getAlto());
    }
}
```

POO. CLASES.

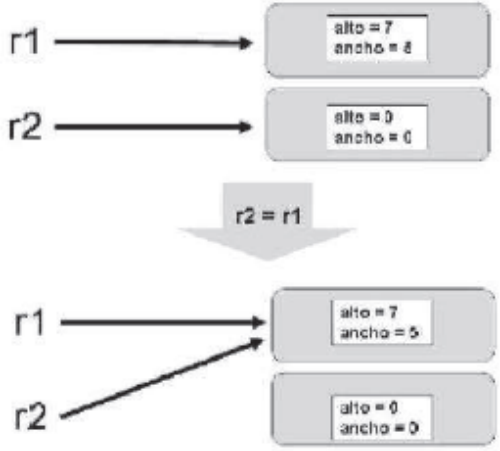
LOS CONSTRUCTORES:

5.2 ASIGNACIÓN DE OBJETOS.

La pregunta es la siguiente: ¿Qué mostrará el programa por pantalla? Tenemos dos opciones:

Opción 1	Opción 2
Alto: 7 Ancho: 5	Alto: 8 Ancho: 6

La respuesta es la **opción 2**. Esto es así porque cuando se hace r2 = r1 se copia la referencia al objeto y no el contenido de un objeto en otro. En la siguiente figura se puede ver esto de forma gráfica:



POO. CLASES.

LOS CONSTRUCTORES:

5.3 CONSTRUCTOR DE COPIA.

¿cómo se copia el contenido de un objeto a otro?

Con un constructor de copia se inicializa un objeto asignándole los valores de otro objeto diferente de la misma clase. Este constructor de copia tendrá solo un parámetro: un objeto de la misma clase.

Un constructor de copia para nuestra clase anterior rectángulo sería el siguiente:

```
Rectangulo(Rectangulo r){ancho= r.getAncho(); alto = r.getAlto();}
```

En el caso de que tenga este nuevo constructor, puedo hacer uso del mismo cuando cree un objeto r2 de la misma clase

POO. CLASES.

LOS CONSTRUCTORES:

5.3 CONSTRUCTOR DE COPIA.

```
public static void main(String[] args){
    Rectangulo r1 = new Rectangulo( anc: 5, alt: 7);
    Rectangulo r2 = new Rectangulo(r1);
    r2.incrementarAlto();
    r2.incrementarAncho();
    System.out.println("Ancho= "+r1.getAncho());
    System.out.println("Alto= "+r1.getAlto());
}
```

La utilización del constructor copia “copiará” los miembros del objeto r1 al objeto r2. El programa anterior ahora mostrará por pantalla lo siguiente:

Alto: 7

Ancho: 5

POO. CLASES.

LOS DESTRUCTORES:

En Java no existen los destructores.

Aunque parezca raro empezar un apartado de destructores diciendo que no existen los destructores, en realidad es así. Al contrario que en C++ y otros lenguajes OO, en Java no existen los destructores. En un intento de simplificar las cosas y mejorar la gestión de memoria, es el propio sistema el que se encarga de eliminar definitivamente los objetos de la memoria cuando le asignamos el valor null a la referencia (referencia = null;), le asignamos a la referencia un objeto diferente o bien termina el bloque donde está definida la referencia.

El sistema de liberación de memoria en Java se llama ***garbage collector*** (recolector de basura), este recolector trabaja de forma automática. Como se vio en apartados anteriores se le puede sugerir que se active realizando la llamada System.gc(), pero esta sola no es la única razón para que se active el recolector de basura (se activará cuando él lo decida, generalmente cuando falta memoria).

POO. CLASES.

LOS DESTRUCTORES:

6.1 LOS FINALIZADORES.

Cuando se va a liberar automáticamente la memoria de objetos inservibles, el sistema ejecuta el finalizador de los objetos. El finalizador se caracteriza por no tener valor de retorno ni argumentos, no puede ser static y denominarse finalize(). Un ejemplo de finalizador es el siguiente:

```
protected void finalize(){System.out.println("Adioossss");}
```

Generalmente, los finalizadores se utilizan para liberar memoria, cerrar ficheros, conexiones, etc. Como no se sabe a ciencia cierta cuándo se van a ejecutarlos finalizadores es el recolector de basura el que se encarga de ello, el consejo es que las operaciones de liberación de ciertos recursos se realicen de forma explícita (a mí no se me ocurriría cerrar una conexión de una base de datos en un finalizador).

Como se verá más adelante, existe una forma de sugerir a Java que ejecute el recolector de basura y es llamando al método System.runFinalization() y luego al recolector de basura System.gc().

POO. CLASES.

LOS DESTRUCTORES:

6.1 LOS FINALIZADORES.

Un ejemplo de la llamada al finalizador es el siguiente:

```
public class rectangulo
{
    ..........
    protected void finalize(){System.out.println("Adioossss");}
    ..........
}
class testfinalize {

    public static void main(String[] args) {
            for (int i = 0; i < 20; i++) {
            rectangulo r = new rectangulo(5,5);
            }
            System.runFinalization();
            System.gc();
    }
}
```

Como se puede observar en el código se ha definido el método finalize() como protected para evitar que pueda ser invocado desde fuera de la clase.

POO. CLASES.

LOS DESTRUCTORES:

6.1 LOS FINALIZADORES.

En el código lo que se ha hecho es crear una serie de objetos cuyo scope o ámbito está reducido a un bucle for. Una vez realizado esto hay que tener en cuenta que finalize() no se invoca cuando termina su scope (cuando termina el bucle). Este método se ejecutará justo antes de ejecutarse el garbage collector por lo que en nuestro código hemos tenido que forzar este hecho con las siguientes líneas:

```
System.runFinalization();
System.gc();
```

Sin las anteriores líneas el programa no mostrará nada por pantalla.

El garbage collector, gc o recolector de basura, se ejecuta en segundo plano en un subproceso paralelo a la propia aplicación. La llamada al recolector de basura se hace ejecutando el método gc() de la clase System.

POO. CLASES.

ENCAPSULACIÓN Y VISIBILIDAD: INTERFACES

Una interfaz es un grupo de métodos con sus cuerpos vacíos. Por ejemplo, la interfaz figura (*intfigura*) podría ser el siguiente:

```
public interface intfigura{
    int area();
}
```

La interfaz define el método área, para su posterior desarrollo en las clases que implementen esta interfaz. Una de las clases que podría implementar esta interfaz es la clase rectángulo:

```
public class rectangulo implements intfigura{
    private int ancho;
    private int alto;
    rectangulo (int an, int al){
            this.ancho = an;
            this.alto = al;
    }

    public int area(){ return ancho*alto; }
}
```

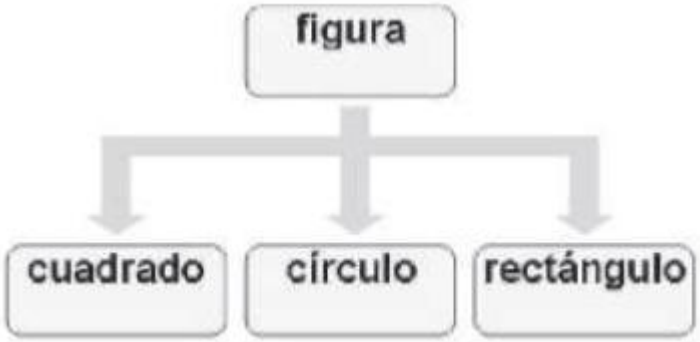
Para compilar correctamente una clase que implementa una interfaz, ésta debe contener los métodos declarados en dicha interfaz.

POO. CLASES.

HERENCIA

La herencia es la base de la reutilización del código. Cuando una clase deriva de una clase padre, ésta hereda todos los miembros y métodos de su antecesor. También es posible redefinir (override) los miembros para adaptarlos a la nueva clase o bien ampliarlos. En general, todas las subclases no solo adoptan las variables y comportamiento de las superclases sino que los amplían.

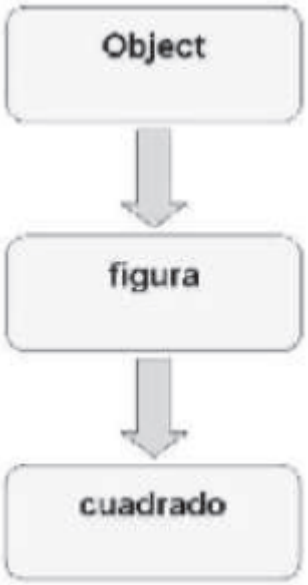
En Java al contrario que en C++ no se permite la herencia múltiple. Es decir, una clase no puede heredar de varias clases.



POO. CLASES.

HERENCIA

Para indicar que una clase hereda de otra se etiqueta con la cláusula extends detrás del nombre de la clase. Por ejemplo, para indicar que la clase rectángulo hereda de la clase figura escribiremos lo siguiente:




```
class rectangulo extends figura { ... }
```

Igual podremos hacer para las demás clases:

```
class circulo extends figura { ... }
class cuadrado extends figura { ... }
```

Todas las clases tienen una superclase o clase padre. Cuando escribas una clase, si ésta no hereda de ninguna clase concreta en realidad hereda de la clase Object (java.lang.Object).

Imaginemos que queremos realizar una estructura de clases como la que se muestra a continuación. El código de la clase figura y cuadrado se muestra junto a la siguiente imagen:

POO. CLASES.

HERENCIA

```
public class figura{
    String color;
    public void setColor(String s){color=s;}
    public String getColor(){return color;}
}

public class cuadrado extends figura{
    private int lado;
    cuadrado(int l){ this.lado = l; }
    public int getArea(){ return lado*lado; }
}
```

Al utilizar la cláusula extends lo que indicamos lo siguiente:

La clase cuadrado es una subclase de la clase figura.

- La clase cuadrado puede utilizar los métodos de la clase figura aunque no estén declarados en la clase cuadrado(siempre y cuando no estén como private en la clase figura).
- Obviamente, los métodos de la subclase no pueden ser utilizados en la superclase o clase principal.

POO. CLASES.

HERENCIA

Las clases heredan el comportamiento de sus antecesores (padres) pero no lo heredan de otras subclases (hermanos).

Imaginemos que queremos testear el comportamiento de la jerarquía anterior. Para ello crearemos la siguiente clase:

```
class testFiguras {
    public static void main(String[] args) {
            cuadrado c=new cuadrado(5);
            c.setColor("Verde");
            System.out.println(c.getColor());
            System.out.println(c.getArea());
    }
}
```

En esta clase se puede observar cómo se llama a métodos de la superclase figura y de la subclase cuadrado. Solamente hemos tenido que crear una clase cuadrado puesto que los atributos y métodos de la clase figura los ha heredado la clase cuadrado.

TEMA 4: PROGRAMACIÓN ORIENTADA A OBJETOS. CLASES